JavaScript Event Loop

The **Event Loop** is the heart of JavaScript.

It is responsible for handling:

- asynchronous operations
- callbacks
- promises
- timers
- DOM events

Without the event loop, JavaScript could only run one line at a time and would freeze during slow tasks.

First Understand This

JavaScript is Single Threaded

JavaScript has:

- ONE call stack
- ONE main thread

Meaning:

At one time, JavaScript can do only ONE thing.

Real Life Example

Imagine:

You are a chef in a small kitchen.

You can do only one task at a time.

Suppose:

1. Customer orders pizza
2. Pizza takes 20 mins to bake
3. Meanwhile another customer orders coffee

If you stand waiting for pizza:

- coffee customer waits
- kitchen becomes slow

Instead:

- put pizza in oven
- continue making coffee

When pizza is ready:

- oven notifies you
- you serve pizza

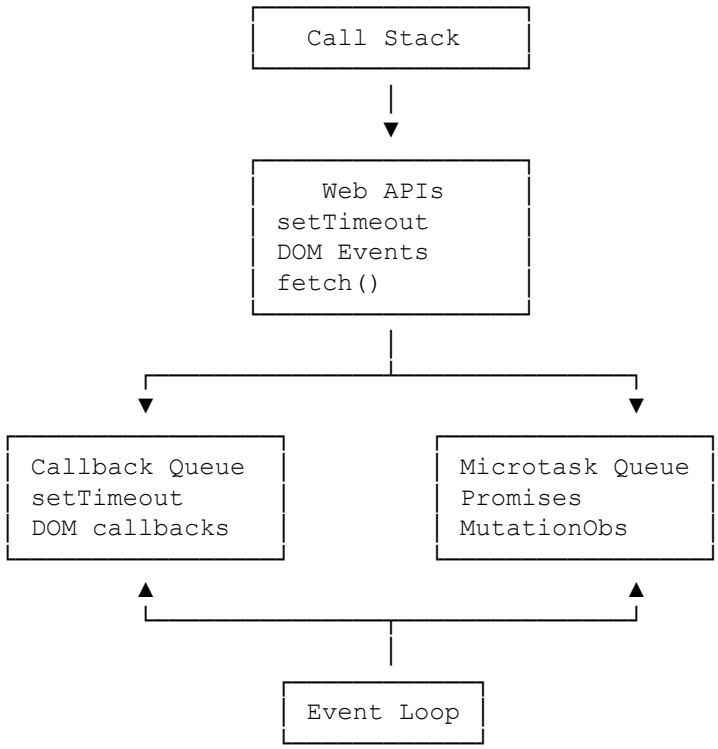
This is exactly how JavaScript works.

Main Components of Event Loop

JavaScript runtime has:

```
1. Call Stack
2. Web APIs
3. Callback Queue
4. Microtask Queue
5. Event Loop
```

Full Diagram



Step 1: Call Stack

The call stack is where JavaScript executes code.

Example:

```
function one() {
```

```
  two();
}

function two() {
  console.log("Hello");
}

one();
```

Stack Execution

```
Call Stack

| two() |
| one() |
| global|
```

After execution:

```
empty
```

Real Life Analogy

Think of plates stacked vertically.

Last added plate comes out first.

This is:

LIFO (Last In First Out)

Problem Without Async System

Suppose:

```
console.log("Start");

setTimeout(() => {
  console.log("Done");
}, 5000);

console.log("End");
```

If JavaScript waited 5 seconds on stack:

Everything freezes.

So browser provides:

Web APIs

Step 2: Web APIs

Provided by browser/runtime.

Examples:

- setTimeout
- DOM
- fetch
- geolocation

These run outside JS engine.

Execution Flow Example

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 2000);

console.log("End");
```

Step-by-Step Internal Working

Step 1

```
console.log("Start");
```

Goes to call stack.

Prints:

```
Start
```

Stack becomes empty.

Step 2

```
setTimeout(...)
```

`setTimeout` enters stack.

Browser sees timer.

Timer sent to Web APIs.

Stack immediately removes setTimeout.

Step 3

```
console.log("End");
```

Prints:

```
End
```

Current Situation

```
Call Stack → Empty
Timer → Running in Web APIs
```

After 2 Seconds

Timer completes.

Callback moves to:

Callback Queue

```
Callback Queue:
----------------
console.log("Timeout")
```

Step 4: Event Loop

Event loop continuously checks:

```
Is call stack empty?
```

If YES:

Move callback from queue to stack.

Final Output

```
Start
End
Timeout
```

Important Concept

setTimeout does NOT guarantee exact timing

It means:

```
minimum delay
```

If stack busy:

callback waits.

Example

```
setTimeout(() => {
  console.log("Timeout");
}, 0);

for (let i = 0; i < 1000000000; i++) {}

console.log("Finished");
```

Output:

```
Finished
Timeout
```

Because stack was busy.

Callback Queue

Stores:

- setTimeout callbacks
- setInterval callbacks
- DOM event callbacks

Also called:

Task Queue / Macrotask Queue

Microtask Queue

Higher priority queue.

Contains:

- Promise callbacks
- queueMicrotask
- MutationObserver

MOST IMPORTANT INTERVIEW QUESTION

Microtask Queue vs Callback Queue

Priority:

```
1. Call Stack
2. Microtask Queue
3. Callback Queue
```

Microtasks execute FIRST.

Example

```
console.log("Start");

setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});

console.log("End");
```

Output

```
Start
End
Promise
Timeout
```

Internal Flow

Step 1

```
Start
```

Step 2

```
setTimeout
```

Sent to Web APIs.

Step 3

Promise callback goes to:

```
Microtask Queue
```

Step 4

```
End
```

Queues Situation

```
Microtask Queue:
```

```
Promise callback

Callback Queue:
Timeout callback
```

Event Loop Priority

Event loop checks:

1. stack empty?
2. execute ALL microtasks
3. then callback queue

So:

`Promise`

comes before:

`Timeout`

Deep Important Rule

After every completed task:

Event loop empties entire microtask queue before moving to callback queue.

Example

```
Promise.resolve().then(() => {
  console.log(1);
});

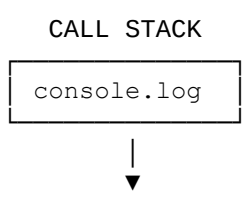
Promise.resolve().then(() => {
  console.log(2);
});

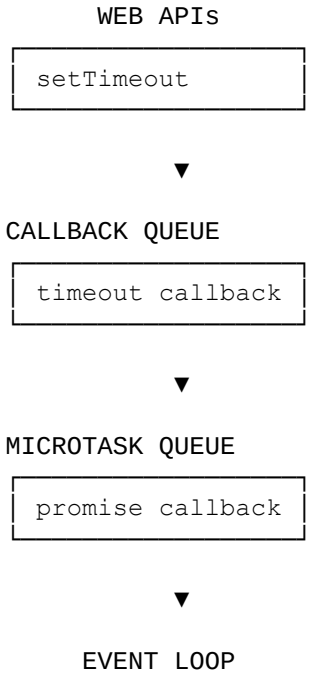
setTimeout(() => {
  console.log(3);
}, 0);
```

Output:

```
1
2
3
```

Visual Diagram of Promise Example





---

fetch() and Event Loop

```
fetch("api")
  .then((res) => res.json())
  .then((data) => {
    console.log(data);
  });
```

Flow:

1. fetch sent to browser API
2. network request runs
3. response returns
4. `.then()` callback goes to microtask queue
5. event loop executes it

DOM Events Example

```
button.addEventListener("click", () => {
  console.log("Clicked");
});
```

Browser waits.

When clicked:

callback enters callback queue.

Infinite Microtask Problem

```
function run() {
```

```
  Promise.resolve().then(run);
}

run();
```

This creates:

infinite microtasks.

Callback queue never executes.

Browser freezes.

Event Loop in Node.js

Node.js also has event loop.

But extra phases exist:

```
Timers
Pending callbacks
Poll
Check
Close callbacks
```

Node uses:

libuv

Common Interview Questions

Why JavaScript is called Single Threaded?

Because it has:

```
one call stack
```

Then how async works?

Using:

- Web APIs
- callback queue
- event loop

Difference Between Stack and Queue

Stack	Queue

Stack	Queue
LIFO	FIFO
last in first out	first in first out

Difference Between Callback Queue and Microtask Queue

Callback Queue	Microtask Queue
lower priority	higher priority
setTimeout	promises
DOM events	queueMicrotask

Why Promise Executes Before setTimeout?

Because promises go to:

```
Microtask Queue
```

which has higher priority.

Complete Advanced Example

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});

console.log("4");
```

Execution Order

Stack

```
1
```

---

setTimeout

Moves to Web APIs.

Promise

Moves to microtask queue.

Stack

4

Stack Empty

Event loop checks microtask queue first.

3

Then callback queue:

2

Final Output

```
1
4
3
2
```

Ultimate Mental Model

Think like this:

```
JavaScript executes synchronous code first.

Async tasks go outside JS engine.

When completed:
- promises → microtask queue
- timers/events → callback queue

Event loop moves tasks back to stack.
```

Final Summary

Component	Work
Call Stack	executes functions
Web APIs	handles async tasks
Callback Queue	stores timer/event callbacks
Microtask Queue	stores promise callbacks
Event Loop	moves tasks to stack

Golden Rule

```
Synchronous Code
→ Microtasks
```

→ Callback Queue

This is the core of JavaScript async behavior.